

CO5 - AT 1
Pseudocode
writing

P. Deepika.p
192525026

1. Social Network Graph - connected components
& Short connection.

Pseudocode

ALGORITHM Social Network (G)

Visited $\leftarrow$ emptySet.

for each user u in G

if u not in visited

BF S(G, u)

print "New connected component"

end if

End for

Shortest $\leftarrow$ DIJKSTRA (G, source)

End ALGORITHM.

2. Task Dependencies - Technologies sorting.

Pseudocode:

ALGORITHM Task Scheduling (G)

calculate indegree of every task

Queue $\leftarrow$ all tasks with indegree 0

Order $\leftarrow$ empty list.


```
while Queue is not empty  
   $u \leftarrow$  remove task from Queue  
  Add  $u$  to Order
```

```
  For each  $v$  adjacent to  $u$ 
```

```
     $\text{indegree}[v] \leftarrow \text{indegree}[v] - 1$ 
```

```
    If  $\text{indegree}[v] = 0$ 
```

```
      Add  $v$  to Queue
```

```
    End if
```

```
  End for
```

```
End while
```

3)

Task Scheduling system

ALGORITHM ScheduleTasks (Tasks, Dependencies)

Create directed graph G

For each dependency ($A \rightarrow B$)

Add edge $A \rightarrow B$

End for :

order $\rightarrow$ Topological - sort(G)

If order is empty

Print "cyclic dependency exists"

Else

Print "valid execution order : ", order

End if

End algorithm

4) DFS with cycle Detection

Pseudocode:

ALGORITHM DFS (u, v)

visited [v] $\leftarrow$ TRUE

recursion stack [v] $\rightarrow$ TRUE

For each v adjacent to u

if visited [v] = FALSE

if DFS(G, v) = TRUE

RETURN TRUE

and if

else if recursion stack [v] = TRUE

return TRUE

end if

End for

Recursion stack [v] $\leftarrow$ FALSE

Return FALSE

End algorithm.

5) Delivery System - Shortest Route.

ALGORITHM DELIVERDUE ($n, source$)

For each vertex n

distance [v] $\leftarrow \infty$

and for

distance [$source$] $\leftarrow 0$

Priority queue $\leftarrow source$

While priority queue is not empty

$U \leftarrow$ vertex with minimum distance

For each edge (u, v) with weight w
new distance $\leftarrow \text{distance}[u] + w$

If new distance $< \text{distance}[v]$

distance $[v] \leftarrow \text{new distance}$,

Insert v into Priority Queue

end if

end for.

end while

Print distance.

end algorithm

for each edge (u, v) with weight w
new distance $\leftarrow \text{distance}[u] + w$

6)

Kruskal's Algorithm - minimum spanning tree
Algorithm Kruskal (a)

MST $\leftarrow$ empty set

sort all edges by increasing weight
create separate set for every vertex

for each edge (u, v, w) in sorted edges

If $\text{find}(u) \neq \text{find}(v)$

Add edge (u, v) to MST

union (u, v)

end if

If number of MST edges $= V - 1$

Print MST


```

else
    Print "Graph is not connected"
end if
End ALGORITHM

```

7.

Floyd-warshall - All-pairs shortest path

ALGORITHM Floydwarshall (v, v)

```

For  $k \leftarrow 1$  to  $v$ 
  For  $i \leftarrow 1$  to  $v$ 
    For  $j \leftarrow 1$  to  $v$ 
       $D[i][j] \leftarrow \min [D[i][j], D[i][k] + D[k][j]]$ 
    end for
  end for
For  $i \leftarrow 1$  to  $v$ 
  If  $[D[i][i] < 0]$ 
    Print "Negative cycle Detected"
  Return  $D$ 
END ALGORITHM.

```

8.

Modified Dijkstra - Avoid Certain Nodes

ALGORITHM Floydwarshall (v, v)

for $k = 1$ to v

for $i = 1$ to v

for $j = 1$ to v

$D[i][j] = \min [D[i][j], D[i][k] + D[k][j]]$


```

End for
End for
End for
For  $i = 1$  to  $v$ 
  If  $D[i][j] < 0$ 
    Print "negative cycle"
  end if
end for
End algorithm.

```

9. Priority Topological sort.

```

Algorithm topological sort( $n$ )
  Calculate indegree of all nodes
  Add all nodes with indegree 0 to priority
  Queue
  While Priority Queue is not empty
     $U =$  Remove highest - priority node
    Print  $U$ 
    For each  $v$  adjacent to  $U$ 
       $\text{indegree}[v] = 0$ 
      Add  $v$  to priority queue
    End if
  End for
End while
End algorithm.

```


10. Modified Dijkstra - void nodes

Algorithm modified Dijkstra (U, S, D, avoid),

Set distance (S) = 0

Set all others distance = ∞

while nodes remain

 v = node with minimum distance.

 If v is in avoid

 Skip v

 end if

 for each neighbour v.

 Update shortest distance to v

 end for

end while

Print shortest path from s to t
algorithm.